

Questions a judge will ask, and how to answer them

Grouped by what the judge is really probing. Each answer gives the fact, the number where there is one, and where on screen to point. The hard questions are in here on purpose — the ones where the honest answer is "no" or "it is a floor" — because those are the ones that decide whether the panel trusts the rest.

Before the demo: open kreaton-upi.vercel.app, press Watch it run, let it reach 400+ payments, filter to "Only the ones it stopped", click the top row. That one screen answers half of these.

Every number here is from the committed artefacts and can be reproduced with `npm run artefacts`.

A. What is this, and why does it matter?

Q What problem are you solving? Banks already have fraud detection.

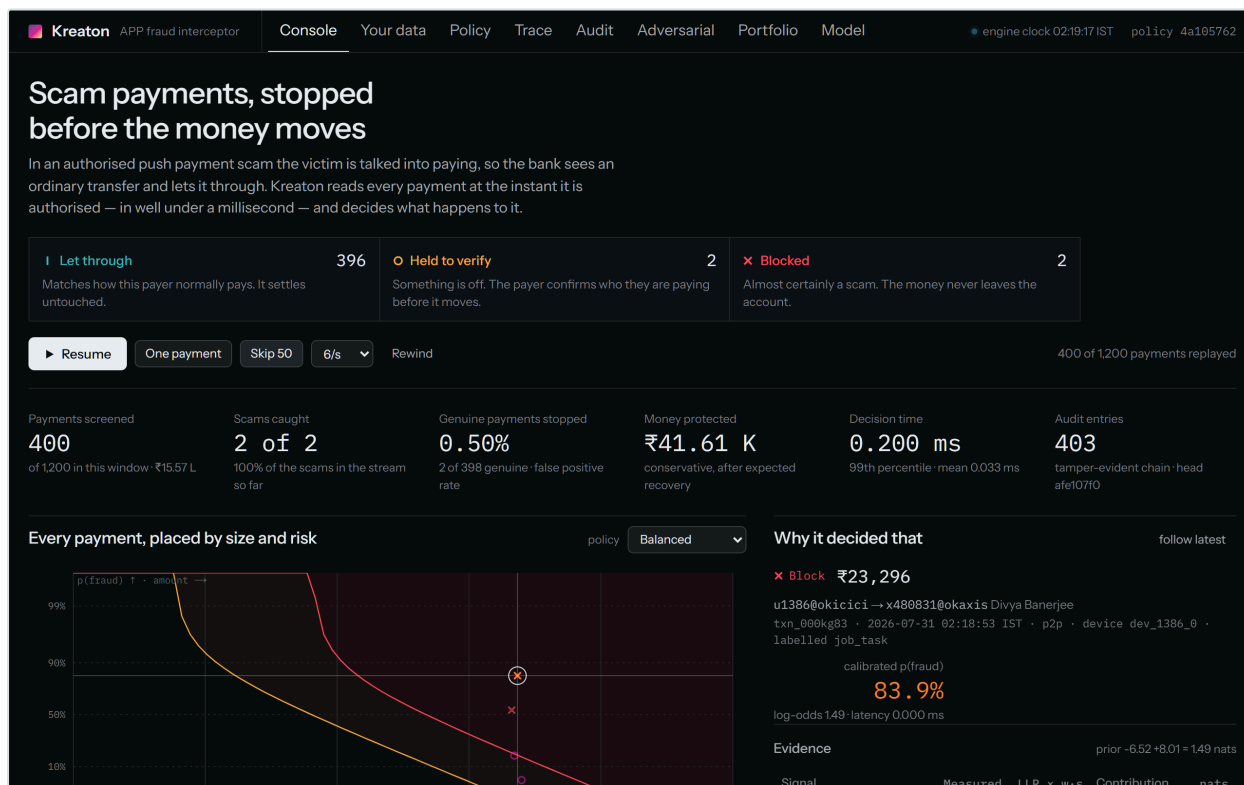
They have fraud detection for *unauthorised* fraud — a stolen card, a phished password. That works because the victim did not make the payment. In an **authorised push payment scam** the victim did: they were talked into it by someone posing as police, a bank officer or an employer, and they pressed the buttons themselves with their own PIN on their own phone. Every authentication check passes, because it really is them. The bank sees an ordinary transfer and lets it through.

What is wrong is the *intent*, and intent leaves a different fingerprint: a beneficiary added minutes ago by pasting an identifier from a chat, a payer on a forty-minute call moving faster than usual, an amount far outside their history, a receiving account that is days old and forwards everything it receives within the hour. Kreation reads that fingerprint at the moment of authorisation and decides.

Point at: the orientation band at the top of the console, which says this in three sentences.

Q Show me it working.

Press **Watch it run**. Payments flow at six a second; the "Let through" counter races ahead while "Held" and "Blocked" barely move — that imbalance is the whole problem in one picture. Press **Only the ones it stopped**, click the top row, and the panel fills in with every signal, its measurement, and how much it moved the verdict. Then inject **Digital arrest** and watch it get blocked at ~97%.



The console 400 payments in. Everything the panel will ask about is on this screen: the three outcomes, the six statistics, the ribbon, the assessment.

Q Is this real data?

No, and we say so on every screen. There is no public dataset of UPI payments labelled for APP fraud with session context; the nearest public material, PaySim, has no call state, no device, no entry method. So the corpus is generated — 4.78 lakh payments, 4,000 payers, eight named scam typologies, each with a written fingerprint. The generator is treated as part of the model: its assumptions are documented, its outputs are checked for leakage, and a guard fails the build if any single signal separates the classes on its own, because on generated data that is how a leak presents.

The *schema* is a real UPI push payment — VPAs, PSP handles, paise amounts, device and SIM hashes, session context. The *values* are synthetic. No figure we quote is a claim about live UPI traffic.

If pressed: "Which would you rather we did — measure honestly on data we generated and documented, or quote numbers we cannot justify?" Then show them the **Your data** screen and offer to run their file.

B. How does the model work?

Q What algorithm is it? Is it machine learning?

It is a fitted statistical model, chosen so that every decision can be reconstructed by hand. Twelve signals are each discretised and mapped to a fitted log-likelihood ratio — the weight-of-evidence form used in credit scoring for decades. Those are weighted, discounted for correlation within their group, floored where an attacker controls the evidence, and summed in log-odds. Isotonic regression maps the sum to a calibrated probability. The weights are fitted by penalised logistic regression (IRLS, ridge centred on one); the calibration by pool-adjacent-violators.

It is *not* a gradient-boosted tree or a neural network, and that was a decision. An additive model is an explanation, not a summary: the contributions sum exactly to the score, and the engine asserts that on every decision. A feature-attribution method laid over a black box approximates; this reconstructs. It also runs in tens of microseconds, which is why the console can execute the real engine in the browser tab.

Point at: the Evidence table in the assessment panel. "Prior $-6.52 + 8.02 = 1.49$ nats" at its top is the sum being shown to add up.

Q What are the twelve signals?

Five groups. **Receiving side:** beneficiary novelty, account age, fan-in $\times$ onward velocity, proximity to a confirmed mule. **Behavioural:** amount deviation against the payer's own history, time-of-day surprisal, velocity burst, drain ratio. **Session context:** concurrent coercion indicators (call, screen share, remote access), urgency (app switches, entry method, seconds to authorise). **Device:** trust age, collapsed to zero on a SIM swap. **Structuring:** placement below limits, window totals crossing them, split re-attempts against an open hold.

The one worth explaining is **fan-in $\times$ onward velocity**. Many unrelated payers in a day describes a popular merchant as well as a mule. What separates them is dwell time: a merchant settles on a cycle; a mule exists to forward. The product isolates the conjunction that identifies a collection account.

Point at: the Model page, which lists every signal with its weight, bin edges, reason-code prefix and a paragraph of rationale.

Q Where does the threshold come from? Who picked it?

Nobody. There is no threshold. Each of the three actions is priced in rupees — approve costs the expected liability (probability $\times$ amount $\times$ unrecovered share $\times$ liability share); hold and block cost friction (handling, abandonment, lifetime value at risk) plus whatever leaks past them — and the cheapest is taken. The boundary falls out of that arithmetic and *moves with the amount*: wrongly approving ₹5 lakh costs far more than wrongly approving ₹500, while wrongly holding either costs about the same. That is why the decision lines on the ribbon slope downward.

Point at: "Expected cost of each action" under the evidence, then the ribbon's two curved lines. Change the policy dropdown to Liability-first and watch the lines move.

Expected cost of each action			margin ₹6,018 over runner-up
Action	Liability	Friction	Total
I Approve	₹18,326	₹0	₹18,326
O Hold	₹6,414	₹86	₹6,500
✗ Block cheapest	₹0	₹482	₹482

Liability = $p \times \text{amount} \times \text{unrecovered share} \times \text{liability share}$. Recovery at the report lag of 240 min is 6.21%.

Three actions, priced. Block is cheapest at ₹482 against ₹18,326 for approving; this is the decision.

Q How accurate is it? Give me numbers.

On the held-out window — 1.91 lakh payments the model never saw during fitting, chronologically after the training window:

- ROC AUC **0.996**, PR AUC **0.890** at 0.30% prevalence, calibration error **0.0019**
- Recall **93.4%** at a 1% false-positive ceiling; **82.3%** at 0.1%
- Under the default policy: **80.9%** of scams caught by count, **95.7%** by value, interrupting **0.145%** of genuine payments at **74.8%** precision
- ₹8.9 crore of liability avoided against ₹3.3 lakh of friction on the 40-day window
- p99 latency **0.118 ms**

On synthetic data. The Portfolio page shows all of it with a per-typology breakdown.

Q Why should I believe your statistics? Did you check them?

Independently, in a different language. `analysis/crosscheck.py` reproduces the fit in scikit-learn and SciPy from the exported feature stream: the per-signal weights agree to five decimal places and sklearn's isotonic regression reaches the same calibration error. Ten of ten checks pass.

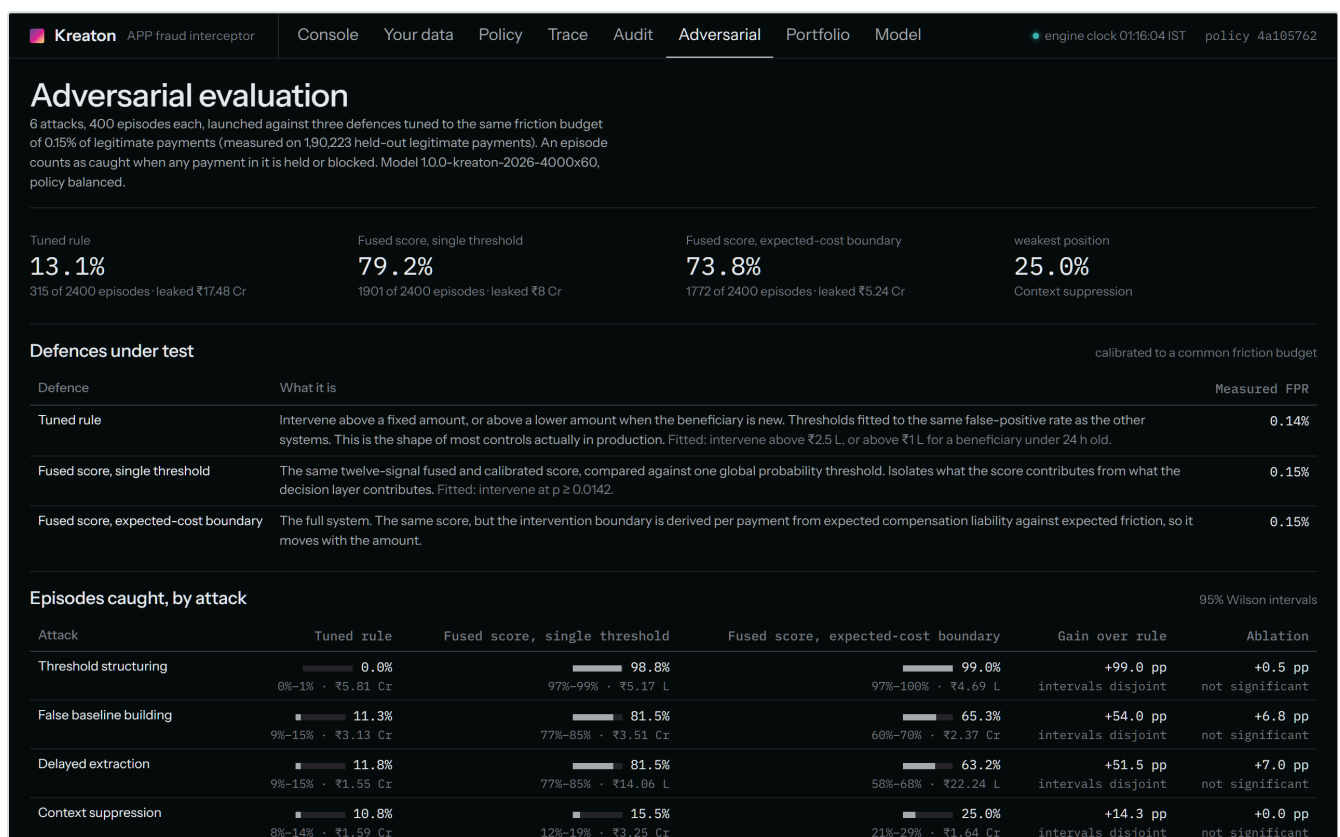
That cross-check is also what found our worst bug. The first committed calibration was nearly a straight line because the isotonic fitter never merged equal levels — sklearn reached an ECE of 0.0018 against our 0.0144. Fixing it moved the false-positive rate from 3.14% to 0.145% at the same detection. We record that in the build notes because the finding is the evidence that the check is real.

C. What happens when the fraudster adapts?

Q A fraudster will learn your rules. Then what?

We ran that. Six attack strategies — structuring below limits, building a false baseline with small payments first, delayed extraction, context hygiene (turning off every indicator the attacker controls), threshold probing, mule rotation — 400 episodes each, against three defences tuned to interrupt the *same* number of genuine customers so the comparison is fair. At matched friction the full engine catches **73.8%** of attacked episodes against **13.1%** for a tuned amount-threshold rule; under the liability-first preset, **81.1%** against **19.3%**.

Point at: the Adversarial page.



Six attacks, three defences, matched friction. The weakest cell is on this page on purpose.

Q What is it worst at?

Context suppression — an attacker who keeps the victim off the phone, has them type the identifier rather than paste it, and tells them to take their time. That removes the signal group that most distinguishes APP fraud from ordinary payments. Detection there is 25% under the balanced policy and 37.5% under liability-first. It is an injectable scenario in the console so the panel can see it fail.

What stops it being zero is the **asymmetric evidence cap**. A fitted model learns that the absence of a call is strong evidence of legitimacy — and the attacker controls that. So the total exculpatory contribution from attacker-controllable signals is floored. The hygienic attacker still has to beat the receiving-side signals, which they do not control: the account's age, its fan-in, its onward velocity.

Say it before they ask: "A fraud tool that only demonstrates its successes is not telling you what it is worth."

Q What if they split the payment to stay under the limit?

The structuring signal looks at the trailing window, not the payment. It fires when the window's total crosses a monitored threshold that no single payment crosses, and adds a large fixed penalty for a re-attempt against an open hold, so halving the amount does not defeat the hold protocol. Inject "Structuring" and "Split re-attempt": the first payment passes, the second is caught, and the retry is linked to the hold and escalated.

Q What if they make a few small payments first to look normal?

The personal baseline uses the median and the median absolute deviation in log space, not the mean and standard deviation. MAD has a fifty-percent breakdown point: one large payment cannot shift it. That attack is "baseline building" in the suite.

D. Can it use our data?

Q Can we run our own dataset through it?

Yes, four ways, all through one importer. Drop a CSV, TSV or JSON file on the **Your data** screen; call `POST /api/v1/import`; run `npm run ingest` for a whole dataset with full metrics; or compose a single payment by hand. A bank statement works — no payer column and no beneficiary number are both handled — and so does PaySim.

Kreation APP fraud interceptor Console **Your data** Policy Trace Audit Adversarial Portfolio Model engine clock 01:16:04 IST policy 4a185762

Your own data

The console ships with a slice of a generated corpus. You can replace it with a file of your own, or compose a single payment and watch the decision move as you change it. Both run through the same authorisation path as everything else.

Bank statement (sample)
5 columns · delimited with ","

Layout

A passbook or account statement: a date, a narration, and separate debit and credit columns.

Which column is which

Anything left unsupplied gets the substitute named beside it. Substitutes are not neutral — the ones marked ● *quiet* hand the engine the reading an attacker would want.

WHO PAID WHOM

Payment reference Unique reference for the payment. Used as the key everywhere in the console.	Not supplied	Numbered from the row position, imp_0000001 upward.
Payer Who is paying. Every behavioural signal is computed against this identity, so it must be stable across rows.	Not supplied	
Payer UPI ID The payer UPI handle, when the dataset has one.	Not supplied	Derived from the payer identifier as <payer>@imported.
Beneficiary Who is being paid. The payer-graph signals key on this, so a dataset that never repeats a The engine starts from nothing: no prior profiles, no prior intelligence. The first 50,000 payments are replayed.	Not supplied	

The importer proposes which column feeds which engine field; the operator confirms every line before anything is scored.

Q What will it tell us about our data that we do not want to hear?

That it almost certainly lacks session context. A bank statement does not know whether the payer was on a call. When those columns are absent the engine does not treat them as neutral — it sees every payment as though the attacker had suppressed every indicator they control. That is the model's weakest position, and a detection rate measured on such a file is a **floor**. The report says so beside the number, names every field left at an attacker-favourable value, every substitution, every dropped row, and the structural limits of the dataset. An unlabelled file gets no accuracy figure at all — decisions and reasons, not a score.

What this file can and cannot show

from the rows read so far

Payments	Payers	Beneficiaries	Ground truth
7	1	5	none
of 8 rows	median 7 payments each	29% repeat	accuracy cannot be scored

This file carries no session context, so the engine sees every payment as if the attacker had suppressed every indicator they control: no call, no urgency, a typed identifier, a trusted device. That is the hardest position for the model and the weakest one in the adversarial report. Detection here is a floor, not a representative figure.

No beneficiary onward-velocity column, so the fan-in signal cannot separate a mule from a busy merchant. That signal is the most specific mule indicator the engine has.

Payers have a median of 7 payments each. Behavioural baselines need history; with this little, amount deviation and time-of-day surprisal are being measured against almost nothing.

No ground-truth column, so the run reports what the engine decided but cannot report whether it was right.

Assumed: No payer column, so every row is read as the same account holder. Behavioural baselines are built for that one payer. No beneficiary identifier column, so beneficiaries are identified by their name.

▶ 12 fields left at the value an attacker would choose

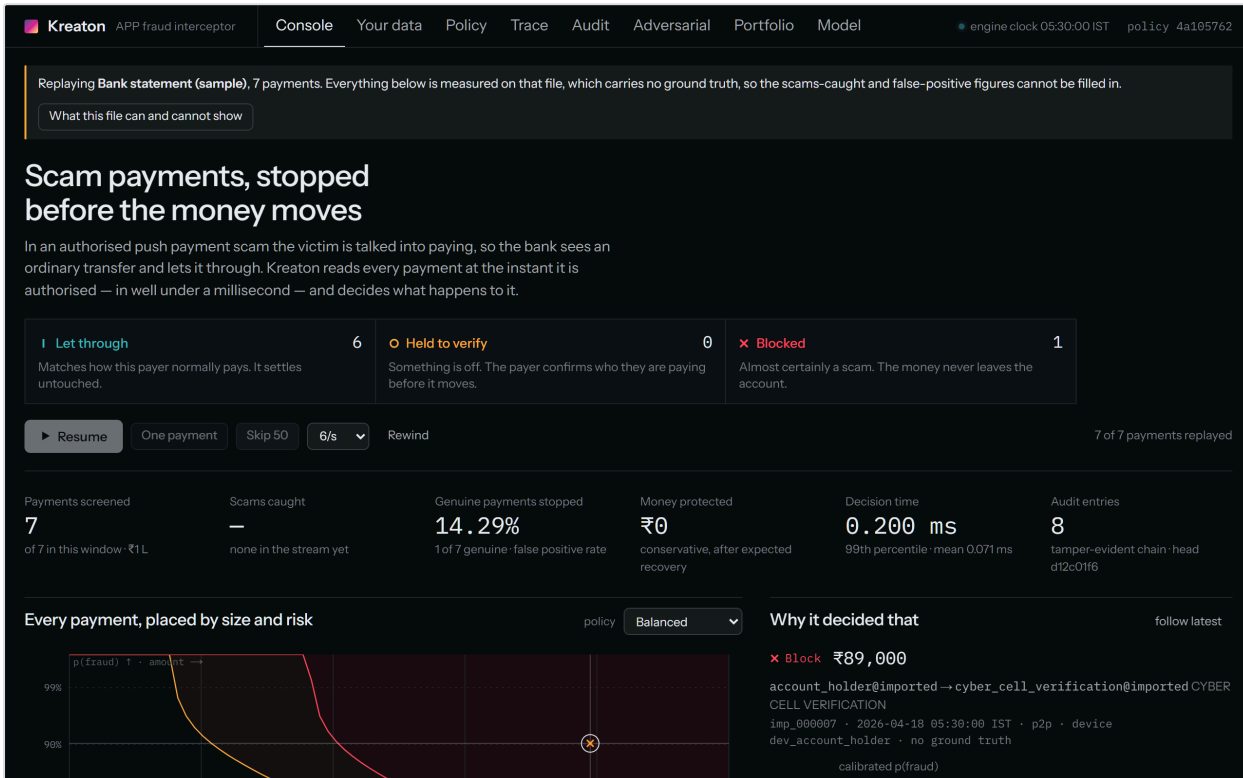
▶ 8 fields filled in harmlessly

[Replay this file](#) The engine starts from nothing: no prior profiles, no prior intelligence. The first 50,000 payments are replayed.

"What this file can and cannot show." The cautions are in plain language and cannot be skipped.

Q Show me it on a bank statement.

The built-in sample is a ten-row passbook: salary in, groceries, electricity, a shop paid three times under three spellings, and one ₹89,000 payment to "CYBER CELL VERIFICATION". With no session context at all the engine blocks that payment at 90.4% — on the receiving account's age, the amount against this account's history, and the drain ratio, which used the statement's own balance column and saw it would empty the account.



The banner says which file is loaded and that it carries no ground truth. The ₹89,000 payment is blocked.

Q Does it connect to real payments? Can it stop a real transfer?

No. Nothing is connected to NPCI, a switch, or any provider sandbox, and nothing moves money. What exists is the integration surface a payment service provider would call in its authorisation path: `POST /api/v1/authorize` takes a payment and returns a decision with the full assessment in about a tenth of a millisecond. Connecting it is an integration project with a PSP, not a change to the engine.

Do not soften this. A judge who hears "sort of" will assume the rest is "sort of" too.

E. Can a regulator trust it?

Q A customer complains their payment was blocked. What do you show them?

The assessment: every signal, what it measured in natural units, how much it moved the verdict, and a stable reason code for each — APP-AMT-7, APP-CAL-4 — that a regulator can be given a table for. Then the three costs and why block was cheapest. The contributions sum exactly to the score; a reviewer who disagrees can point at the line that produced it.

Point at: the assessment panel; open the trace for the full case file.

Q How do you prove nobody altered a decision afterwards?

Every decision is written to an append-only ledger where each record carries a SHA-256 of the one before it. Change any record and every later hash stops matching; the gate verifies the chain, exports it, re-imports it, tampers with one record and checks the break is detected at exactly that record. The Audit page is the regulator's view — what was decided, when, under which model digest and which policy digest.

The screenshot shows the 'Audit' page of the 'Kreation APP fraud interceptor'. The page has a dark theme and a navigation bar at the top with tabs: Console, Your data, Policy, Trace, Audit (selected), Adversarial, Portfolio, and Model. On the right of the navigation bar, it shows 'engine clock 01:16:04 IST' and 'policy 4a185762'.

Compliance trail

Append-only, hash-chained. Each record commits to the digest of the one before it, so altering any entry invalidates every later digest and the verifier reports where.

Buttons: Verify chain, Tamper a copy, Export JSONL

records sealed
1

head digest
b8feabb6f97646ed
commits to the whole history

verification
not run
recomputes every digest

tamper check
not run
alters one record in a copy

Query

filters are conjunctive

Transaction id
txn_...

Record kind
any

Decision
any

Reason code
APP-CALL-3

Minimum p(fraud), percent
1

1 record match

Records

newest first, up to 200

Seq	Time	Kind	Summary	Digest
0	09:49:32	MODEL_LOADED	1.00-kreation-2026-4000x60 096614f59c00 fitted 2026-09-11 2026-09-12 09:49:32 IST	b8feabb6f976

The audit trail. Policy changes are on the same chain as decisions, so a change of settings is itself evidence.

Q Where is that record kept? What happens when the server restarts?

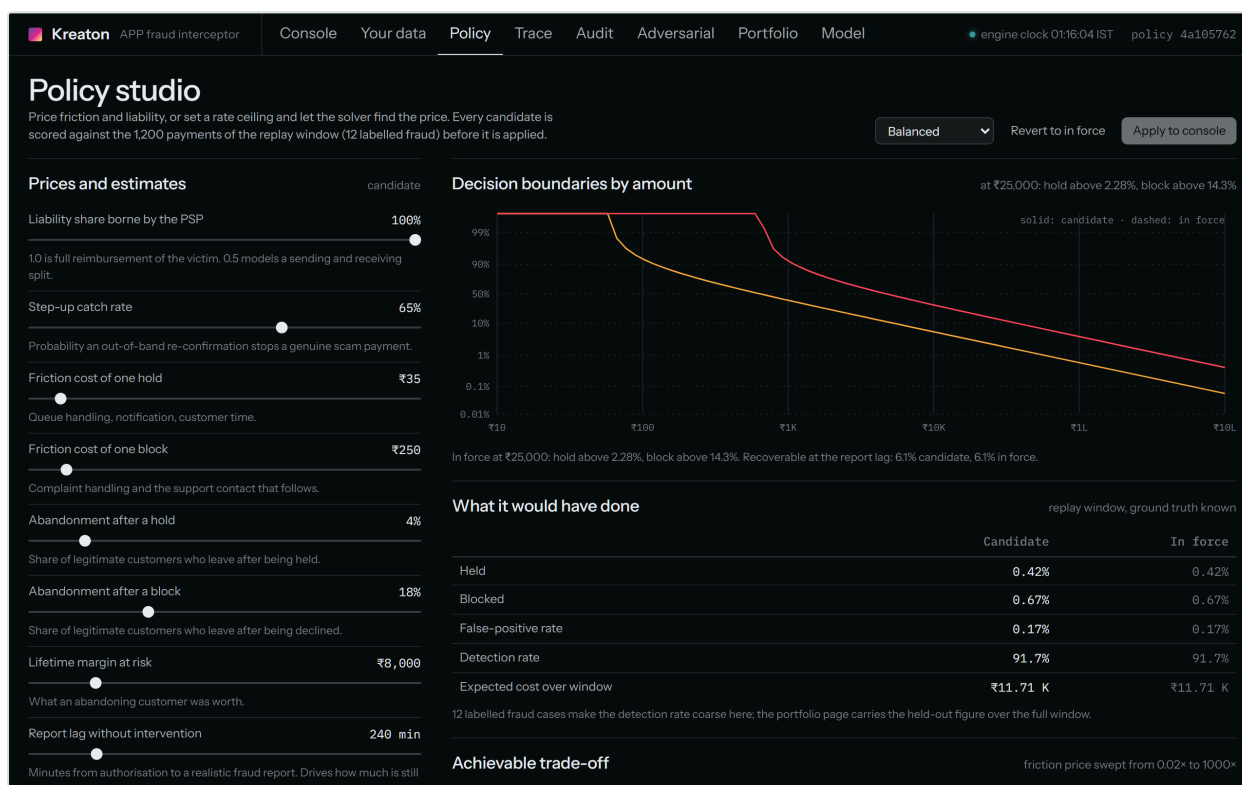
Hot state is in memory, because twelve signals each doing a database round trip would blow a millisecond budget — every production fraud engine works this way. Durability sits *behind* the decision: after each response is returned, the sealed record, the payment and any hold are copied in batches to a Google Sheet through a service account. A judge can open that spreadsheet in a browser. If the copy fails the decision is unaffected, and `/api/v1/health` shows the failure count and the last error rather than pretending a record is being kept.

We verified it against the live site: a decision made at `kreaton-upi.vercel.app` was read back out of the sheet.

If asked "a spreadsheet?": at this scale, yes, and it has one property no database has — anyone can read it without credentials or a query tool. At portfolio volume the sink is the seam to replace; it is five optional methods.

Q Who decides the trade-off between stopping scams and annoying customers?

The institution, and the Policy page is where. It is expressed two ways: as prices — liability share, friction per hold, abandonment rate, lifetime value — for a risk officer, and as rate ceilings — "no more than 0.5% of genuine payments interrupted" — for an operations team, solved as a shadow price of friction. Every candidate is priced against the scored population before it is applied, and applying it is written to the ledger.



The policy studio. The frontier chart shows what each setting buys and costs.

F. How was it built?

Q What is the stack?

One TypeScript monorepo. The engine (`packages/core`) has **zero runtime dependencies** and runs identically in a browser tab and a serverless function. The console is Next.js 16 with React 19, plain CSS, and hand-drawn SVG charts — no UI library, no chart library. Simulation, fitting and the adversarial suite are TypeScript run with `tsx` . A Python sidecar cross-checks the statistics offline. Vitest for 107 tests. GitHub Actions drives the Vercel CLI to deploy.

Q Why no ML framework? Why no database?

Because the requirements — reconstructible by hand, microsecond latency, runs in a browser — are met better without them, and every dependency is something a reviewer has to trust. The fitter is 500 lines of IRLS and isotonic regression we can point at. The store is a synchronous interface with a memory implementation because a real-time decision path cannot wait on a network; durability is a sink behind it.

Q How is it tested?

107 unit tests, and a sixteen-check model quality gate that generates a reduced corpus, fits, replays and asserts on discrimination, calibration, the single-signal leak guard, the operating point, p99 latency, the audit chain and tamper detection, and the recoverability curve. Both run on every push. Nothing deploys unless both pass.

Q What went wrong during the build?

Seven things worth naming, all in `docs/PROGRESS.md` . The calibration bug (found by the cross-check). CI failing on every commit because a generated type only existed locally. Three separate fixes to the deploy smoke test, each findable only by running it. A deploy that failed after adding two packages because Vercel builds inside `apps/web` and never compiled them. A Sheets test suite that passed 21 tests then wrote empty columns in production, because the fixture had invented a shape the engine never emits. Flushes that never completed on Vercel because the instance freezes after the response. And the console saying "labelled legitimate" for payments a file never labelled.

We record these because a build log with no failures in it was not watched.

G. Rapid-fire

QUESTION	ANSWER
How fast is a decision?	Mean 0.04 ms, p99 0.12 ms, for twelve signals, fusion, calibration, recoverability, three costs and a sealed audit record.
How many scams does it miss?	19.1% by count on the held-out window, 4.3% by value. The ones it misses are small.
How many customers does it annoy?	1.45 in a thousand under the default policy. The Policy page lets you make it fewer, and shows what that costs.
What is "held to verify"?	A soft hold with a challenge chosen so the coach on the phone cannot relay it — a named-payee confirmation, not an OTP. Re-attempts are linked; a budget of retries then escalates to human review.
What is the recoverability curve?	How much of the money could still be frozen at a given delay, from a Monte Carlo over mule-chain hops. It falls to 6% by four hours — which is why stopping a payment beats chasing it.
Why is the "approve" colour teal, not green?	Green against the block red failed colour-blind separation (deutan ΔE 7.6). Teal at hue 195 is the worst pair at 13.5.
Does it work on a phone?	Yes. Every route verified free of horizontal overflow at 390px.
What happens on reload?	The shipped replay resets. An imported file and the replay position are kept in the browser and restored.
Can I see the code?	github.com/junekashif/Kreaton , public, MIT. Every figure regenerates with <code>npm run artefacts</code> .
Who built it?	Team Kreaton.
What would you do next?	Run PaySim's real file. Connect a PSP sandbox. Replace the sink with Postgres at volume. Get a beneficiary-intelligence feed, which is the input the payee-graph group depends on and the one no sending institution can observe alone.

H. The demo, in order

1. **Open** `kreaton-upi.vercel.app`. Say what APP fraud is in two sentences. (30s)
2. **Watch it run**. Let the counters diverge. Name the imbalance. (30s)
3. **Filter** to "Only the ones it stopped". Click the top row. Walk the evidence table top to bottom; point at the sum. Point at the three costs. (90s)
4. **Inject Digital arrest**. Blocked at ~97%. Then **inject Context suppression** and let it pass. Say why. Switch to Liability-first, inject again, watch it hold. (90s)

5. **Your data.** Load the bank statement sample, show the report, replay it, show the ₹89,000 block with every session signal quiet. (90s)
6. **Adversarial** page: 73.8% vs 13.1%. **Audit** page: the chain. (45s)
7. **Close** on what is not built: real rails, real data. Offer to run their file. (30s)

Built by Team Kreation for COGNITIA 2026, problem statement FINTECH-PS2. Screenshots taken from the live console on 12 September 2026. All payments, payers and scams shown are synthetic.